Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040074
Nama	Try Noer Arreva
Kelas	C
Link Github Repository	tokyooolie/PW2026_243040073

Bagian 1. Screenshot Semua Kode
1. App.jsx

```css
.wadah {
  max-width: 640px;
  margin: 40px auto;
  padding: 0 20px;
  font-family: Arial, sans-serif;
  color: #1f2937;
}
.judul {
  color: #1b365d;
  margin-bottom: 4px;
}
.subjudul {
  color: #6b7280;
  margin-top: 45px;
  margin-bottom: 24px;
}
.kartu {
  background: #ffffff;
  border: 1px solid #d0d7de;
  border-radius: 10px;
  padding: 20px;
  margin-bottom: 20px;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0,
0.08);
}
.kartu h2 {
  margin-top: 0;
  font-size: 18px;
  color: #1b365d;
}
.baris {
  display: flex;
  gap: 8px;
}
.input {
  flex: 1;
  padding: 10px;
  border: 1px solid #d0d7de;
  border-radius: 6px;
  font-size: 14px;
}
.tombol {
  padding: 10px 16px;
  border: none;
  border-radius: 6px;
  background: #1b365d;
  color: #ffffff;
  font-weight: bold;
  cursor: pointer;
}
```

```css
.tombol:disabled {
  background: #9ca3af;
  cursor: not-allowed;
}
.tombol-merah {
  background: #dc2626;
}
.tombol-kecil {
  padding: 6px 10px;
  font-size: 12px;
}
.daftar {
  list-style: none;
  padding: 0;
  margin: 0;
}
.item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 0;
  border-bottom: 1px solid #e5e7eb;
}
.item span {
  flex: 1;
}
.selesai {
  text-decoration: line-through;
  color: #9ca3af;
}
.kosong {
  color: #6b7280;
  text-align: center;
  font-style: italic;
}
.galat {
  color: #dc2626;
  font-size: 13px;
  margin: 6px 0 0;
}
.ringkasan {
  font-size: 14px;
  color: #374151;
}

/* FILTER TUGAS */
.filter {
  display: flex;
  gap: 8px;
  margin: 18px 0;
  flex-wrap: wrap;
}

.filter button {
  padding: 9px 14px;
  border: 1px solid #1b365d;
  border-radius: 6px;
  background: white;
  color: #1b365d;
  cursor: pointer;
}
```



```css
.saldo-card {
  background: #1b365d;
  color: white;
  padding: 24px;
  border-radius: 10px;
  margin-bottom: 15px;
}

.saldo-card h3 {
  margin: 0 0 12px;
  font-size: 16px;
  font-weight: normal;
}

.saldo-card h2 {
  color: white;
  margin: 0;
  font-size: 28px;
}

.saldo-detail {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
}

.saldo-detail div {
  background: white;
  border: 1px solid #d0d7de;
  padding: 15px;
  border-radius: 8px;
  min-width: 0;
}

.saldo-detail p {
  color: #6b7280;
  font-size: 13px;
  margin-bottom: 8px;
}

.pemasukan {
  color: #16a34a;
}

.pengeluaran {
  color: #dc2626;
}

.transaksi {
  justify-content: space-between;
}

.transaksi div {
  min-width: 0;
  overflow-wrap: anywhere;
}

.transaksi p {
  margin-top: 5px;
  font-size: 13px;
}

@media (max-width: 480px) {
  .baris {
    flex-wrap: wrap;
  }

  .saldo-detail {
    grid-template-columns: 1fr;
  }

  .saldo-card h2 {
    font-size: 23px;
  }
}
.ringkasan-keuangan {
  margin-bottom: 0;
}

.saldo-card {
  text-align: center;
  background: #f3f6fa;
  color: #1b365d;
  border: 1px solid #d0d7de;
}

.saldo-card h2 {
  color: #1b365d;
}

.saldo-detail {
  margin-top: 12px;
}

.saldo-detail div {
  text-align: center;
  background: #ffffff;
}
```

2. App.jsx

```jsx
import { useState, useEffect } from "react";
import "./App.css";

import Kartu from "./Kartu";
import Penghitung from "./Penghitung";
import FormTugas from "./FormTugas";
import ItemTugas from "./ItemTugas";
import FormKeuangan from "./FormKeuangan";
import DaftarTransaksi from "./DaftarTransaksi";
import RingkasanSaldo from "./RingkasanSaldo";

function App() {
  // STATE DAFTAR TUGAS
  const [daftarTugas, setDaftarTugas] =
useState([
    { id: 1, judul: "Install Node.js dan Vite",
selesai: true },
    { id: 2, judul: "Pelajari useState",
selesai: false },
  ]);

  // STATE FILTER
  const [filter, setFilter] = useState("semua");

  // MENAMBAHKAN TUGAS
  const tambahTugas = (judul) => {
    const tugasBaru = {
      id: Date.now(),
      judul,
      selesai: false,
    };

    setDaftarTugas([...daftarTugas, tugasBaru]);
  };

  // MENGUBAH STATUS TUGAS
  const ubahStatus = (id) => {
    const daftarBaru = daftarTugas.map((tugas)
=> {
      if (tugas.id === id) {
        return { ...tugas, selesai:
!tugas.selesai };
      }
      return tugas;
    });

    setDaftarTugas(daftarBaru);
  };

  // MENGHAPUS SATU TUGAS
  const hapusTugas = (id) => {
    setDaftarTugas(daftarTugas.filter((tugas) =>
tugas.id !== id));
  };

  // MENGHAPUS SEMUA TUGAS SELESAI
  const hapusSemuaSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas) =>
!tugas.selesai));
  };
```



```
  // JUMLAH TUGAS SELESAI
    const jumlahSelesai =
daftarTugas.filter((tugas) =>
tugas.selesai).length;

    // FILTER DAFTAR TUGAS
    const tugasDitampilkan =
daftarTugas.filter((tugas) => {
      if (filter === "belum") return
!tugas.selesai;
      if (filter === "selesai") return
tugas.selesai;
      return true;
    });

    // STATE CATATAN KEUANGAN
    const [transaksi, setTransaksi] = useState(()
=> {
      try {
        return
JSON.parse(localStorage.getItem("transaksi")) ||
[];
      } catch {
        return [];
      }
    });

    // SIMPAN TRANSAKSI KE LOCAL STORAGE
    useEffect(() => {
      localStorage.setItem("transaksi",
JSON.stringify(transaksi));
    }, [transaksi]);

    // MENAMBAHKAN TRANSAKSI
    const tambahTransaksi = (data) => {
      setTransaksi([...transaksi, { ...data, id:
Date.now() }]);
    };

    // MENGHAPUS TRANSAKSI
    const hapusTransaksi = (id) => {
      setTransaksi(transaksi.filter((item) =>
item.id !== id));
    };

    return (
      <div className="wadah">
        <h1 className="judul">Daftar Tugas
Praktikum</h1>

        <p className="subjudul">
          Laboratorium Informatika - Modul 2:
React Lanjutan
        </p>

        <Penghitung />

        <Kartu judul="Tambah Tugas Baru">
          <FormTugas onTambah={tambahTugas} />
        </Kartu>

        <Kartu judul="Daftar Tugas">
          <p className="ringkasan">
            {jumlahSelesai} dari
{daftarTugas.length} tugas selesai
          </p>

          {/* TOMBOL FILTER */}
          <div className="filter">
            <button
              className={filter === "semua" ?
"filter-aktif" : ""}
              onClick={() => setFilter("semua")}
            >
              Semua
            </button>

            <button
              className={filter === "belum" ?
"filter-aktif" : ""}
              onClick={() => setFilter("belum")}
            >
              Belum Selesai
            </button>

            <button
              className={filter === "selesai" ?
"filter-aktif" : ""}
              onClick={() => setFilter("selesai")}
            >
              Selesai
            </button>
          </div>

          {/* HAPUS SEMUA TUGAS SELESAI */}
          {jumlahSelesai > 0 && (
            <button
              className="tombol tombol-merah
hapus-semua"
              onClick={hapusSemuaSelesai}
            >
              Hapus Semua yang Selesai
            </button>
          )}
```

```
 {/* DAFTAR TUGAS */}
        {tugasDitampilkan.length === 0 ? (
          <p className="kosong">Tidak ada tugas
pada kategori ini.</p>
        ) : (
          <ul className="daftar">
            {tugasDitampilkan.map((tugas) => (
              <ItemTugas
                key={tugas.id}
                tugas={tugas}
                onUbahStatus={ubahStatus}
                onHapus={hapusTugas}
              />
            ))}
          </ul>
        )}
      </Kartu>

      {/* APLIKASI CATATAN KEUANGAN MINI */}
      <h1 className="judul">Catatan Keuangan
Mini</h1>

      <p className="subjudul">Aplikasi
Pencatatan Pemasukan dan Pengeluaran</p>

     <Kartu judul="Ringkasan Keuangan">
  <RingkasanSaldo transaksi={transaksi} />
</Kartu>

<Kartu judul="Tambah Transaksi">
        <FormKeuangan onTambah={tambahTransaksi}
/>
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        <DaftarTransaksi transaksi={transaksi}
onHapus={hapusTransaksi} />
      </Kartu>
    </div>
  );
}

export default App;
```

3. DaftarTransaksi,jsx

```jsx
function DaftarTransaksi({ transaksi, onHapus })
{
  const formatRupiah = (angka) => {
    return new Intl.NumberFormat("id-ID", {
      style: "currency",
      currency: "IDR",
      maximumFractionDigits: 0,
    }).format(angka);
  };

  if (transaksi.length === 0) {
    return (
      <p className="kosong">
        Belum ada transaksi. Silakan tambahkan
transaksi!
      </p>
    );
  }

  return (
    <ul className="daftar">
      {transaksi.map((item) => (
        <li className="item transaksi" key=
{item.id}>
          <div>
            <strong>{item.keterangan}</strong>
            <p
              className={
                item.jenis === "pemasukan" ?
"pemasukan" : "pengeluaran"
              }
            >
              {item.jenis === "pemasukan" ?
"Pemasukan" : "Pengeluaran"}:{" "}
              {formatRupiah(item.nominal)}
            </p>
          </div>

          <button
            className="tombol tombol-merah
tombol-kecil"
            onClick={() => onHapus(item.id)}
          >
            Hapus
          </button>
        </li>
      ))}
    </ul>
  );
}

export default DaftarTransaksi;
```

4. FormKeuangan.jsx

```jsx
import { useState } from "react";

function FormKeuangan({ onTambah }) {
  const [keterangan, setKeterangan] =
useState("");
  const [nominal, setNominal] = useState("");
  const [jenis, setJenis] =
useState("pemasukan");

  const tanganiKirim = (event) => {
    event.preventDefault();

    if (!keterangan.trim() || Number(nominal) <=
0) {
      return;
    }

    onTambah({
      keterangan: keterangan.trim(),
      nominal: Number(nominal),
      jenis,
    });

    setKeterangan("");
    setNominal("");
    setJenis("pemasukan");
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="form-keuangan">
        <label>Keterangan</label>
        <input
          className="input"
          type="text"
          placeholder="Contoh: Uang makan"
          value={keterangan}
          onChange={(e) =>
setKeterangan(e.target.value)}
        />

        <label>Nominal (Rp)</label>
        <input
          className="input"
          type="number"
          placeholder="Masukkan nominal"
          min="1"
          value={nominal}
          onChange={(e) =>
setNominal(e.target.value)}
        />

        <label>Jenis Transaksi</label>
        <select
          className="input"
          value={jenis}
          onChange={(e) =>
setJenis(e.target.value)}
        >
          <option
value="pemasukan">Pemasukan</option>
          <option
value="pengeluaran">Pengeluaran</option>
        </select>

        <button className="tombol"
type="submit">
          Tambah Transaksi
        </button>
      </div>
    </form>
  );
}

export default FormKeuangan;
```

5. FormTugas.jsx

```jsx
import { useState } from "react";

function FormKeuangan({ onTambah }) {
  const [keterangan, setKeterangan] =
useState("");
  const [nominal, setNominal] = useState("");
  const [jenis, setJenis] =
useState("pemasukan");

  const tanganiKirim = (event) => {
    event.preventDefault();

    if (!keterangan.trim() || Number(nominal) <=
0) {
      return;
    }

    onTambah({
      keterangan: keterangan.trim(),
      nominal: Number(nominal),
      jenis,
    });

    setKeterangan("");
    setNominal("");
    setJenis("pemasukan");
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="form-keuangan">
        <label>Keterangan</label>
        <input
          className="input"
          type="text"
          placeholder="Contoh: Uang makan"
          value={keterangan}
          onChange={(e) =>
setKeterangan(e.target.value)}
        />

        <label>Nominal (Rp)</label>
        <input
          className="input"
          type="number"
          placeholder="Masukkan nominal"
          min="1"
          value={nominal}
          onChange={(e) =>
setNominal(e.target.value)}
        />

        <label>Jenis Transaksi</label>
        <select
          className="input"
          value={jenis}
          onChange={(e) =>
setJenis(e.target.value)}
        >
          <option
value="pemasukan">Pemasukan</option>
          <option
value="pengeluaran">Pengeluaran</option>
        </select>

        <button className="tombol"
type="submit">
          Tambah Transaksi
        </button>
      </div>
    </form>
  );
}

export default FormKeuangan;
```

6. Index.css

```css
:root {
  --text: #6b6375;
  --text-h: #08060d;
  --bg: #fff;
  --border: #e5e4e7;
  --code-bg: #f4f3ec;
  --accent: #aa3bff;
  --accent-bg: rgba(170, 59, 255, 0.1);
  --accent-border: rgba(170, 59, 255, 0.5);
  --social-bg: rgba(244, 243, 236, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0,
0, 0, 0.05) 0 4px 6px -2px;

  --sans: system-ui, "Segoe UI", Roboto, sans-
serif;
  --heading: system-ui, "Segoe UI", Roboto,
sans-serif;
  --mono: ui-monospace, Consolas, monospace;

  font: 18px/145% var(--sans);
  letter-spacing: 0.18px;
  color-scheme: light dark;
  color: var(--text);
  background: var(--bg);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;

  @media (max-width: 1024px) {
    font-size: 16px;
  }
}

@media (prefers-color-scheme: dark) {
  :root {
    --text: #9ca3af;
    --text-h: #f3f4f6;
    --bg: #16171d;
    --border: #2e303a;
    --code-bg: #1f2028;
    --accent: #c084fc;
    --accent-bg: rgba(192, 132, 252, 0.15);
    --accent-border: rgba(192, 132, 252, 0.5);
    --social-bg: rgba(47, 48, 58, 0.5);
    --shadow:
      rgba(0, 0, 0, 0.4) 0 10px 15px -3px,
rgba(0, 0, 0, 0.25) 0 4px 6px -2px;
  }

  #social .button-icon {
    filter: invert(1) brightness(2);
  }
}
```



```css
body {
  margin: 0;
}

#root {
  width: 1126px;
  max-width: 100%;
  margin: 0 auto;
  text-align: center;
  border-inline: 1px solid var(--border);
  min-height: 100svh;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
}

h1,
h2 {
  font-family: var(--heading);
  font-weight: 500;
  color: var(--text-h);
}

h1 {
  font-size: 56px;
  letter-spacing: -1.68px;
  margin: 32px 0;
  @media (max-width: 1024px) {
    font-size: 36px;
    margin: 20px 0;
  }
}
h2 {
  font-size: 24px;
  line-height: 118%;
  letter-spacing: -0.24px;
  margin: 0 0 8px;
  @media (max-width: 1024px) {
    font-size: 20px;
  }
}
p {
  margin: 0;
}

code,
.counter {
  font-family: var(--mono);
  display: inline-flex;
  border-radius: 4px;
  color: var(--text-h);
}

code {
  font-size: 15px;
  line-height: 135%;
  padding: 4px 8px;
  background: var(--code-bg);
}
```

7. ItemTugas.jsx



```jsx
function ItemTugas({ tugas, onUbahStatus,
onHapus }) {
  return (
    <li className="item">
      <input
        type="checkbox"
        checked={tugas.selesai}
        onChange={() => onUbahStatus(tugas.id)}
      />
      {/* className dinamis: dicoret jika tugas
sudah selesai */}
      <span className={tugas.selesai ? "selesai"
: ""}>{tugas.judul}</span>
      <button
        className="tombol tombol-merah tombol-
kecil"
        onClick={() => onHapus(tugas.id)}
      >
        Hapus
      </button>
    </li>
  );
}
export default ItemTugas;
```

8. Kartu.jsx

```jsx
function Kartu({ judul, children }) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}
export default Kartu;
```

9. Main.jsx

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

10. Penghitung.jsx



```jsx
import { useState } from "react";
import Kartu from "./Kartu";
function Penghitung() {
  // jumlah = nilai saat ini, setJumlah = fungsi
untuk mengubahnya, 0 = nilai awal
  const [jumlah, setJumlah] = useState(0);
  const tambah = () => {
    setJumlah(jumlah + 1);
  };
  const ulangi = () => {
    setJumlah(0);
  };
  return (
    <Kartu judul="Penghitung Klik">
      <p>Tombol sudah diklik {jumlah} kali</p>
      {/* Conditional rendering: pesan ini hanya
muncul jika jumlah >= 5 */}
      {jumlah >= 5 && <p>Wah, kamu rajin sekali
mengklik!</p>}
      <div className="baris">
        <button className="tombol" onClick=
{tambah}>
          Klik Saya
        </button>
        <button className="tombol tombol-merah"
onClick={ulangi}>
          Ulangi
        </button>
      </div>
    </Kartu>
  );
}
export default Penghitung;
```

11. RingkasanSaldo.jsx

```jsx
function RingkasanSaldo({ transaksi }) {
  const totalPemasukan = transaksi
    .filter((item) => item.jenis ===
"pemasukan")
    .reduce((total, item) => total +
item.nominal, 0);

  const totalPengeluaran = transaksi
    .filter((item) => item.jenis ===
"pengeluaran")
    .reduce((total, item) => total +
item.nominal, 0);

  const saldo = totalPemasukan -
totalPengeluaran;

  const formatRupiah = (angka) => {
    return new Intl.NumberFormat("id-ID", {
      style: "currency",
      currency: "IDR",
      maximumFractionDigits: 0,
    }).format(angka);
  };

  return (
    <div className="ringkasan-keuangan">
      <div className="saldo-card">
        <h3>Total Saldo</h3>
        <h2>{formatRupiah(saldo)}</h2>
      </div>

      <div className="saldo-detail">
        <div>
          <p>Total Pemasukan</p>
          <strong className="pemasukan">
{formatRupiah(totalPemasukan)}</strong>
        </div>

        <div>
          <p>Total Pengeluaran</p>
          <strong className="pengeluaran">
            {formatRupiah(totalPengeluaran)}
          </strong>
        </div>
      </div>
    </div>
  );
}

export default RingkasanSaldo;
```

Bagian 2. Penjelasan Kode

1. App.css

File App.css digunakan untuk mengatur tampilan antarmuka aplikasi, seperti ukuran, warna, jarak, dan tata letak setiap komponen. File ini mengatur tampilan kartu, tombol, input, daftar tugas, form keuangan, ringkasan saldo, serta warna pemasukan dan pengeluaran agar aplikasi terlihat lebih rapi dan mudah digunakan.

2. App.jsx

File App.jsx merupakan komponen utama yang mengatur jalannya aplikasi. File ini menggunakan useState untuk mengelola data daftar tugas, filter tugas, dan transaksi keuangan. Selain itu, useEffect digunakan untuk menyimpan data transaksi ke dalam localStorage agar data tidak hilang ketika halaman dimuat ulang.

Komponen ini juga memiliki beberapa fungsi, yaitu:

- tambahTugas() untuk menambahkan tugas baru.
- ubahStatus() untuk mengubah status tugas menjadi selesai atau belum selesai.
- hapusTugas() untuk menghapus tugas tertentu.
- hapusSemuaSelesai() untuk menghapus seluruh tugas yang sudah selesai.
- tambahTransaksi() untuk menambahkan transaksi keuangan.
- hapusTransaksi() untuk menghapus transaksi berdasarkan ID.

Selain itu, terdapat fitur filter untuk menampilkan seluruh tugas, tugas yang belum selesai, dan tugas yang sudah selesai.

3. DaftarTransaksi.jsx

File DaftarTransaksi.jsx digunakan untuk menampilkan seluruh data transaksi keuangan yang telah dimasukkan oleh pengguna. Komponen ini menggunakan map() untuk menampilkan daftar transaksi dan formatRupiah() untuk mengubah nominal menjadi format mata uang Rupiah. Warna transaksi juga dibedakan menggunakan class dinamis, yaitu hijau untuk pemasukan dan merah untuk pengeluaran. Tersedia pula tombol Hapus untuk menghapus transaksi tertentu.

4. FormKeuangan.jsx

File FormKeuangan.jsx berfungsi sebagai formulir untuk memasukkan data transaksi keuangan. Formulir ini terdiri dari tiga input, yaitu keterangan, nominal, dan jenis transaksi berupa pemasukan atau pengeluaran.

Komponen ini menggunakan useState untuk mengelola nilai setiap input. Fungsi tanganiKirim() digunakan untuk memproses data transaksi, melakukan validasi agar keterangan tidak kosong dan nominal lebih dari nol, serta mengirimkan data ke komponen utama melalui fungsi onTambah(). Setelah transaksi berhasil ditambahkan, formulir akan dikosongkan kembali.

5. FormTugas.jsx

File FormTugas.jsx digunakan untuk menyediakan formulir penambahan tugas baru. Komponen ini menggunakan useState untuk menyimpan teks yang dimasukkan pengguna. Terdapat validasi agar judul tugas tidak kosong dan tidak melebihi 50 karakter. Fungsi tanganiKirim() digunakan untuk mengirimkan judul tugas ke komponen utama melalui onTambah(). Setelah berhasil, kolom input akan dikosongkan kembali.

6. index.css

File index.css digunakan untuk mengatur tampilan dasar aplikasi secara keseluruhan. Pengaturan di dalamnya meliputi jenis font, warna latar belakang, warna teks, ukuran halaman, serta pengaturan elemen utama body dan #root. File ini menjadi dasar tampilan sebelum styling khusus diterapkan melalui App.css.

7. ItemTugas.jsx

File ItemTugas.jsx digunakan untuk menampilkan satu item tugas dalam daftar tugas. Komponen ini menampilkan judul tugas, checkbox untuk mengubah status penyelesaian, dan tombol Hapus. Checkbox menggunakan properti checked dan fungsi onUbahStatus() untuk mengubah status tugas. Apabila tugas telah selesai, class selesai digunakan untuk memberikan efek garis coret pada judul tugas.

8. Kartu.jsx

File Kartu.jsx merupakan komponen yang dapat digunakan kembali (*reusable component*) untuk membungkus konten aplikasi dalam bentuk kartu. Komponen ini menerima properti judul dan children. Properti judul digunakan untuk menampilkan judul kartu, sedangkan children digunakan untuk menampilkan isi sesuai dengan komponen yang ditempatkan di dalamnya.

9. main.jsx

File main.jsx merupakan titik awal (*entry point*) aplikasi React. File ini menggunakan ReactDOM.createRoot() untuk menghubungkan komponen utama App.jsx dengan elemen HTML yang memiliki ID root. Selain itu, file ini mengimpor index.css agar pengaturan tampilan global diterapkan ketika aplikasi dijalankan.

10. Penghitung.jsx

File Penghitung.jsx digunakan untuk membuat fitur penghitung jumlah klik. Komponen ini menggunakan useState dengan nilai awal 0 untuk menyimpan jumlah klik. Fungsi tambah() digunakan untuk menambah jumlah klik, sedangkan fungsi ulangi() digunakan untuk mengembalikan jumlah klik menjadi 0. Terdapat juga *conditional rendering* untuk menampilkan pesan khusus apabila jumlah klik telah mencapai lima kali atau lebih.

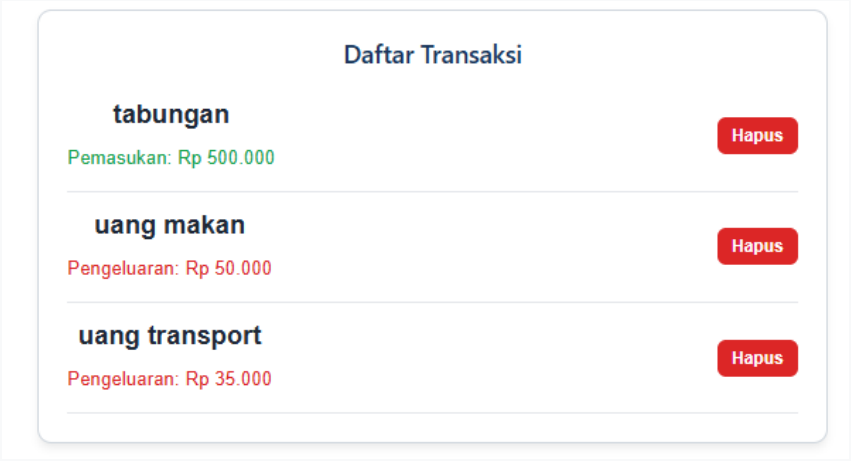
11. RingkasanSaldo.jsx

File RingkasanSaldo.jsx digunakan untuk menghitung dan menampilkan ringkasan keuangan pengguna. Komponen ini menggunakan filter() untuk memisahkan transaksi pemasukan dan pengeluaran, kemudian menggunakan reduce() untuk menjumlahkan nominal masing-masing transaksi. Total saldo diperoleh dari hasil pengurangan total pemasukan dengan total pengeluaran. Hasil perhitungan ditampilkan dalam format Rupiah menggunakan Intl.NumberFormat().

Bagian 3. Screenshot Hasil Antarmuka

1. 

2. 

3. 

Bagian 4. Penjelasan Hasil Antarmuka

1. Tampilan Ringkasan Keuangan

Pada tampilan ini, aplikasi menampilkan halaman utama Catatan Keuangan Mini yang berisi ringkasan kondisi keuangan pengguna. Terdapat informasi Total Saldo sebesar Rp 415.000, Total Pemasukan sebesar Rp 500.000, dan Total Pengeluaran sebesar Rp 85.000. Nilai saldo dihitung secara otomatis berdasarkan selisih antara total pemasukan dan total pengeluaran. Bagian ini membantu pengguna mengetahui kondisi keuangan secara keseluruhan.

2. Tampilan Form Tambah Transaksi

Pada bagian ini, aplikasi menyediakan formulir untuk menambahkan transaksi keuangan. Formulir terdiri dari kolom keterangan untuk memasukkan nama transaksi, kolom nominal untuk memasukkan jumlah uang, serta pilihan jenis transaksi berupa pemasukan atau pengeluaran. Setelah seluruh data diisi, pengguna dapat menekan tombol Tambah Transaksi untuk menyimpan data ke dalam daftar transaksi.

3. Tampilan Daftar Transaksi

Pada tampilan ini, aplikasi menampilkan seluruh transaksi yang telah berhasil ditambahkan oleh pengguna. Setiap transaksi menampilkan keterangan dan nominal berdasarkan jenis transaksinya. Transaksi pemasukan ditampilkan menggunakan warna hijau, sedangkan transaksi pengeluaran ditampilkan menggunakan warna merah. Selain itu, tersedia tombol Hapus pada setiap transaksi yang berfungsi untuk menghapus data transaksi tertentu dari daftar.